

## COMPUTAÇÃO GRÁFICA

# Exercício prático — Mini ambiente virtual 3D com Python + Pygame

*Aula 5 — Plataformas para criação de ambientes virtuais*

Disciplina: Computação Gráfica / Realidade Aumentada / Realidade Virtual

Proposta de atividade para desenvolvimento individual ou em grupos de 2 a 3 estudantes.

## 1. Apresentação da atividade

Neste exercício, você construirá uma pequena cena tridimensional interativa usando Python e Pygame. A aplicação funcionará como uma plataforma gráfica mínima: terá uma janela, modelos geométricos, transformações, uma câmera virtual, projeção e interação pelo teclado. O objetivo não é substituir uma engine completa, mas tornar visível o fluxo de desenvolvimento estudado na Aula 5 e relacioná-lo ao pipeline gráfico trabalhado nas aulas anteriores.

A cena conterá pelo menos um cubo e uma pirâmide. Cada objeto será definido por vértices e arestas, transformado no espaço e projetado em uma janela 2D. O estudante também poderá movimentar a câmera, alternar a projeção e observar como diferentes componentes de uma plataforma participam da construção de um ambiente virtual.

## 2. Objetivos de aprendizagem

- Reconhecer o papel de uma biblioteca gráfica na construção de uma aplicação interativa.
- Distinguir, na prática, biblioteca, framework e engine, identificando o que precisa ser implementado manualmente no Pygame.
- Modelar objetos simples por meio de vértices e arestas.
- Aplicar transformações lineares e afins: escala, rotação e translação.
- Manipular uma câmera virtual e compreender seu efeito sobre a cena.
- Relacionar janela, laço de eventos, renderização e interação ao ciclo de uma aplicação gráfica.
- Comparar projeção perspectiva e ortográfica e comunicar as decisões tomadas no projeto.

## 3. Conceitos mobilizados

| Conceito           | Como aparece no exercício                                                                                    |
|--------------------|--------------------------------------------------------------------------------------------------------------|
| Plataforma gráfica | O Pygame fornece janela, eventos, desenho 2D e temporização; o estudante implementa a parte 3D simplificada. |
| Cena virtual       | Conjunto de objetos, câmera, fundo e informações de interação.                                               |
| Modelo geométrico  | Lista de vértices e arestas que descreve cubo e pirâmide.                                                    |
| Transformação      | Escala, rotação e translação aplicadas em coordenadas locais.                                                |
| Câmera             | Posição virtual que altera a conversão do mundo para a tela.                                                 |
| Pipeline           | Modelo → transformação → câmera → projeção → rasterização.                                                   |
| Interação          | Eventos e teclas modificam a câmera, a projeção e os objetos.                                                |

## 4. Enunciado

Implemente uma aplicação chamada “Mini ambiente virtual 3D”. A aplicação deverá abrir uma janela de 1000 × 700 pixels e exibir uma cena com, no mínimo, um cubo e uma pirâmide. Os modelos devem ser definidos por dados geométricos, sem desenhar cada aresta manualmente em posições fixas da tela.

A cada quadro, o programa deverá aplicar as transformações dos objetos, considerar a posição da câmera, projetar os pontos tridimensionais e desenhar as arestas na janela. A interação deve permitir explorar a cena e comparar os efeitos da perspectiva com os da projeção ortográfica.

## 5. Requisitos funcionais

- Abrir e fechar corretamente a janela do Pygame.
- Representar um cubo e uma pirâmide por vértices e arestas.
- Organizar cada objeto com posição, rotação, escala e cor.
- Aplicar rotação, escala e translação antes da projeção.
- Implementar projeção perspectiva e ortográfica.
- Permitir deslocar a câmera nos eixos X, Y e Z.
- Permitir rotacionar pelo menos dois objetos.
- Exibir na tela o modo de projeção e os controles.
- Usar um laço principal com processamento de eventos, atualização e renderização.
- Evitar falhas quando um ponto se aproxima do plano de recorte da câmera.

## 6. Instalação e execução

Abra um terminal na pasta em que o programa será salvo e execute os comandos abaixo:

```
python -m venv .venv
# Windows
.venv\Scripts\activate
# Linux ou macOS
source .venv/bin/activate
python -m pip install pygame
python mini_ambiente_virtual.py
```

Se o comando “python” não estiver disponível no seu sistema, tente “python3”. O programa deve ser salvo com o nome `mini_ambiente_virtual.py`.

## 7. Código-base completo

O código a seguir implementa uma solução funcional. Leia cada função, execute o programa e depois modifique a aplicação conforme as propostas de investigação. Os comentários indicam a relação entre as etapas do código e os conceitos da disciplina.

```
import math
import sys
import pygame

pygame.init()
LARGURA, ALTURA = 1000, 700
tela = pygame.display.set_mode((LARGURA, ALTURA))
pygame.display.set_caption("Mini ambiente virtual - Aula 5")
relogio = pygame.time.Clock()
fonte = pygame.font.SysFont("arial", 22)

# Cada objeto e uma lista de vertices e arestas em coordenadas locais.
CUBO = [(-1,-1,-1), (1,-1,-1), (1,1,-1), (-1,1,-1),
        (-1,-1,1), (1,-1,1), (1,1,1), (-1,1,1)]
ARESTAS_CUBO = [(0,1), (1,2), (2,3), (3,0), (4,5), (5,6), (6,7), (7,4),
                (0,4), (1,5), (2,6), (3,7)]

PIRAMIDE = [(-1,-1,-1), (1,-1,-1), (1,-1,1), (-1,-1,1), (0,1,0)]
ARESTAS_PIRAMIDE = [(0,1), (1,2), (2,3), (3,0), (0,4), (1,4), (2,4), (3,4)]

def rotacionar(ponto, angulo_x, angulo_y, angulo_z):
```

```

"""Aplica rotacoes lineares sucessivas nos eixos X, Y e Z."""
x, y, z = ponto
cx, sx = math.cos(angulo_x), math.sin(angulo_x)
y, z = y*cx - z*sx, y*sx + z*cx
cy, sy = math.cos(angulo_y), math.sin(angulo_y)
x, z = x*cy + z*sy, -x*sy + z*cy
cz, sz = math.cos(angulo_z), math.sin(angulo_z)
x, y = x*cz - y*sz, x*sz + y*cz
return (x, y, z)

def transformar(ponto, posicao, rotacao, escala):
    """Modelo: escala, rotacao e translacao no mundo."""
    escalado = tuple(c * escala for c in ponto)
    girado = rotacionar(escalado, *rotacao)
    return tuple(g + t for g, t in zip(girado, posicao))

def projetar(ponto, camera, perspectiva=True):
    """Converte coordenadas do mundo em coordenadas da janela."""
    x, y, z = (ponto[i] - camera[i] for i in range(3))
    z = max(z, 0.15) # plano de recorte simplificado
    fator = 420 / z if perspectiva else 170
    return (int(LARGURA/2 + x*fator), int(ALTURA/2 - y*fator))

def desenhar_objeto(objeto, arestas, configuracao, camera, perspectiva):
    pontos_2d = []
    for vertice in objeto:
        mundo = transformar(vertice, configuracao["posicao"],
                           configuracao["rotacao"], configuracao["escala"])
        pontos_2d.append(projetar(mundo, camera, perspectiva))
    for a, b in arestas:
        pygame.draw.line(tela, configuracao["cor"], pontos_2d[a], pontos_2d[b], 3)

objetos = [
    {"modelo": CUBO, "arestas": ARESTAS_CUBO, "posicao": (-2, 0, 7),
     "rotacao": (0, 0, 0), "escala": 1.2, "cor": (70, 190, 255)},
    {"modelo": PIRAMIDE, "arestas": ARESTAS_PIRAMIDE, "posicao": (2, 0, 8),
     "rotacao": (0, 0, 0), "escala": 1.5, "cor": (255, 170, 70)},
]

camera = [0, 0, 0]
perspectiva = True
executando = True
while executando:
    dt = relógio.tick(60) / 1000
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            executando = False
        if evento.type == pygame.KEYDOWN:
            if evento.key == pygame.K_ESCAPE:
                executando = False
            elif evento.key == pygame.K_p:
                perspectiva = not perspectiva
            elif evento.key == pygame.K_r:
                camera[:] = [0, 0, 0]
                for obj in objetos: obj["rotacao"] = (0, 0, 0)

teclas = pygame.key.get_pressed()

```

```

    velocidade = 3 * dt
    if teclas[pygame.K_LEFT]: camera[0] -= velocidade
    if teclas[pygame.K_RIGHT]: camera[0] += velocidade
    if teclas[pygame.K_UP]: camera[1] += velocidade
    if teclas[pygame.K_DOWN]: camera[1] -= velocidade
    if teclas[pygame.K_w]: camera[2] += velocidade
    if teclas[pygame.K_s]: camera[2] -= velocidade
    if teclas[pygame.K_a]: objetos[0]["rotacao"] = (0, objetos[0]["rotacao"]
[1]+velocidade, 0)
    if teclas[pygame.K_d]: objetos[1]["rotacao"] = (0, objetos[1]["rotacao"][1]-
velocidade, 0)

    tela.fill((18, 24, 38))
    for obj in objetos:
        desenhar_objeto(obj["modelo"], obj["arestas"], obj, camera, perspectiva)
    modo = "perspectiva" if perspectiva else "ortografica"
    texto = fonte.render(f"Projecao: {modo} | P: alternar | R: reiniciar", True,
(235,235,235))
    instrucoes = fonte.render("Setas/W/S: camera | A/D: rotacionar objetos | ESC: sair",
True, (190,205,220))
    tela.blit(texto, (24, 22))
    tela.blit(instrucoes, (24, 52))
    pygame.display.flip()

pygame.quit()
sys.exit()

```

## 8. Controles da aplicação

| Tecla                  | Ação                                      |
|------------------------|-------------------------------------------|
| Setas esquerda/direita | Mover a câmera no eixo X.                 |
| Setas cima/baixo       | Mover a câmera no eixo Y.                 |
| W / S                  | Aproximar ou afastar a câmera no eixo Z.  |
| A                      | Rotacionar o cubo.                        |
| D                      | Rotacionar a pirâmide.                    |
| P                      | Alternar entre perspectiva e ortográfica. |
| R                      | Reiniciar câmera e rotações.              |
| ESC                    | Encerrar a aplicação.                     |

## 9. Roteiro de desenvolvimento e investigação

1. Execute o código sem alterações e identifique os elementos da cena: objetos, câmera, fundo, texto e laço principal.
2. Altere a posição e a escala do cubo. Registre como a transformação muda sua representação na tela.
3. Modifique a rotação para aplicar giros simultâneos em mais de um eixo. Observe que a ordem das rotações influencia o resultado.
4. Movimente a câmera no eixo Z e compare o tamanho aparente dos objetos em perspectiva.
5. Pressione P e repita a observação em projeção ortográfica. Explique a diferença.
6. Adicione um terceiro objeto, por exemplo um prisma ou uma segunda pirâmide, mantendo a mesma estrutura de dados.
7. Crie uma grade simples no plano XZ ou eixos coordenados para facilitar a leitura da cena.
8. Escolha uma melhoria e documente: preenchimento de faces, iluminação simulada, seleção de objetos ou controle por mouse.

## 10. Questões para discussão

9. Quais recursos o Pygame oferece diretamente e quais partes do pipeline precisaram ser programadas?
10. Por que este programa é uma biblioteca gráfica com desenho 2D, e não uma engine 3D completa?
11. Qual é a diferença entre coordenadas locais do modelo, coordenadas do mundo e coordenadas da tela?
12. Como escala, rotação e translação alteram a posição e o tamanho dos objetos?
13. Por que a perspectiva usa a profundidade para alterar o fator de escala?
14. O que acontece quando dois objetos ocupam posições próximas na cena, mas são desenhados sem ordenação por profundidade?
15. Que componentes seriam necessários para transformar esta atividade em uma aplicação XR?
16. Quais vantagens e limitações existem em usar Pygame, Godot, Unity ou Unreal para este tipo de projeto?

## 11. Extensões opcionais

- Implementar preenchimento das faces usando `pygame.draw.polygon` e ordenar as faces pela profundidade média.
- Adicionar uma terceira pessoa ou uma câmera orbital controlada pelo mouse.
- Criar um catálogo de modelos simples e permitir inserir ou remover objetos da cena.
- Adicionar seleção de objetos por clique e destacar o objeto selecionado.
- Implementar uma iluminação simplificada baseada na orientação das faces.
- Criar uma tela inicial com instruções, uma cena principal e uma tela de encerramento, aproximando o programa da estrutura de uma aplicação interativa.
- Comparar o mesmo conceito em uma engine, descrevendo quais recursos seriam fornecidos automaticamente.

## 12. Entrega esperada

Cada estudante ou grupo deverá entregar:

- O programa Python executável.
- Uma captura de tela da cena em projeção perspectiva e outra em projeção ortográfica.
- Uma descrição breve da organização da cena e do pipeline implementado.
- Respostas às questões de discussão.
- Uma proposta de extensão implementada ou um plano detalhado para realizá-la.

## 13. Síntese conceitual

A atividade mostra que uma plataforma de desenvolvimento não é apenas a janela em que o resultado aparece. Ela organiza recursos para representar a cena, receber entradas, atualizar o estado do mundo e produzir uma saída visual. Ao usar Pygame, o estudante percebe quais serviços são fornecidos por uma biblioteca e quais responsabilidades permanecem no código da aplicação. Essa observação prepara a comparação com engines que oferecem editor de cenas, gerenciamento de ativos, câmeras, iluminação, física, scripts e integração com dispositivos XR.

A sequência fundamental observada no exercício é:

**modelagem → transformação → câmera → projeção → rasterização → interação**

Esse fluxo conecta os conteúdos das Aulas 3, 4 e 5 e serve como base para os temas seguintes de manipulação de objetos, interação e desenvolvimento de experiências tridimensionais.